

1 - Introduction

我們將討論什麼是 React、React 是如何工作的以及 React 特性。

什麼是 React？

React 是一個用於建構用戶界面的開源 **JavaScript** 程式庫。

這個定義有兩個關鍵點：

1. **React** 是一個 **JavaScript** 程式庫，而不是一個框架。
2. 構建用戶界面，所以 **React** 並不關注應用程序的其他方面，如路由或 HTTP 請求，它只負責構建豐富的用戶界面。

React 擁有豐富的生態系統，可以與其他程式庫很好地合作，並且能夠構建成熟的 Web 應用程序。

以下是對 **React** 的總結：

- **React** 是一個 **JavaScript** 程式庫。
- **React** 不是一個框架。
- **React** 是 **Facebook** 建立的一個開源專案。
- **React** 用於在前端構建用戶界面 (UI)。
- **React** 是 **MVC** 應用程序（模型檢視控制器）的檢視層

React 最重要的方面之一是您可以建立組件，就像自定義的、可重用的 **HTML** 元素一樣，以快速高效地構建用戶界面。

React 是如何工作的？

React 在記憶體中建立一個 **VIRTUAL DOM** - **React** 不是直接操作瀏覽器的 **DOM**，而是在記憶體中建立一個虛擬 **DOM**，在瀏覽器 **DOM** 中進行更改之前，它會在其中進行所有必要的操作。

React 只更改需要更改的內容 - **React** 找出已進行的更改，並且僅更改需要更改的內容。

JSX - **JSX** 是 **JavaScript** 語法擴充。

在 **React** 開發中不是必須使用 **JSX**，但建議使用。

Component - **React** 您需要將所有內容都視為一個組件。這將幫助您在處理更大規模的專案時維護程式碼。

先決條件

在開始使用 **React** 之前，您應該提前了解一些事情。

以下是我認為是 **React** 的先決條件。

- 基本熟悉 **HTML** 和 **CSS**。

- **JavaScript** 和程式的基礎知識。
- 對 **DOM** 的基本了解。
- 熟悉 **ES6** 語法和特性。
- 全域安裝 **Node.js** 和 **npm** 。

讓我們建一個簡單的 React hello-world 應用程序來理解 React 的動作。

直接在 HTML 中做出反應

開始學習 React 的最快方法是直接在 HTML 文件中編寫 React。

讓我們逐步手動建立 React 應用程序。

第 1 步 - index.html

讓我們建立 index.html 文件並向其中添加以下內容：

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Hello React!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  </head>

  <body>
    <div id="root"></div>
    <script type="text/babel">
      // React code will go here
    </script>
  </body>
</html>
```

請注意，我們在<head>添加了三個 CDN—— React、 React DOM 和 Babel。我們添加了一個名為 root 的 id 的 div，最後我們建了一個<script>標籤，您的自定義程式碼將用於此。

Babel 是 JavaScript 的編譯器(compiler)工具，可以把 ES6 標準以上的新語法轉換成最基本的 JS 語法，另外也支援 react 語法轉換，只要簡單的配置就能完

成，這已是現今非常流行的轉換工具了。**Babel** 搭配一些自動化打包套件還可以加快效率與自動運行，在前端工程中很受歡迎。

請注意，我們使用了以下程式庫：

- **React** - React API
- **React DOM** - 添加特定於 DOM 的方法
- **Babel** - 一個 JavaScript 編譯器，可以讓我們在舊瀏覽器中使用 ES6+

我們應用程序的入口點將是 `<div id="root"></div>` 元素，這是按照約定命名的。您還會注意到 `text/babel` 腳本類型，這是使用 **Babel** 所必需的。

第 2 步 - 建立應用程序組件

讓我們使用 ES6 類來建立一個名為 **App** 的 **React** 組件。

```
class App extends React.Component {  
  //...  
}
```

現在我們將添加 `render()` 方法，這是類組件中唯一必需的方法，用於呈現 DOM 節點。

```
class App extends React.Component {  
  render() {  
    return (  
      //...  
    );  
  }  
}
```

請注意，在返回值中，我們將放置一個看起來像簡單 HTML 元素的內容：

```
class App extends React.Component {  
  render() {  
    return <h1>Hello world!</h1>  
  }  
}
```

第 3 步 - React DOM render() 方法

最後，我們將使用 **React DOM render()** 方法將我們建立的 **App** 類渲染到 HTML 的根 `div` 中。

```
ReactDOM.render(<App />, document.getElementById('root'))
```

index.html 的完整程式碼

這是我們 `index.html` 的完整程式碼：

```

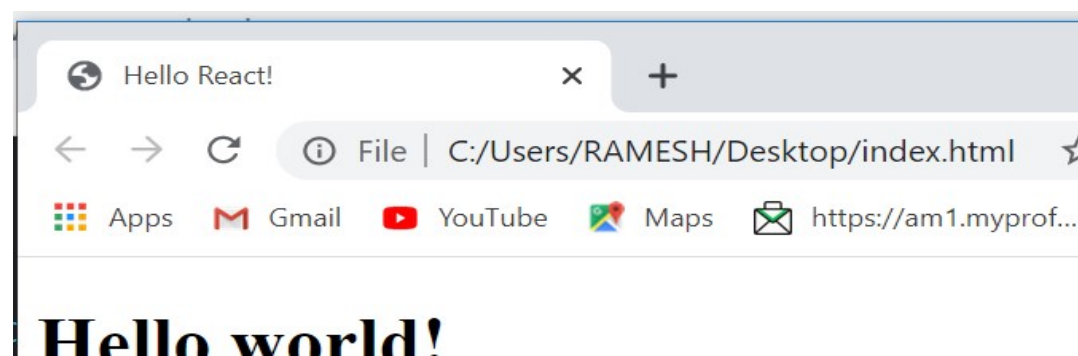
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Hello React!</title>
    <script
src="https://unpkg.com/react@16/umd/react.development.js"></script>
    <script
src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  </head>

  <body>
    <div id="root"></div>
    <script type="text/babel">
      class App extends React.Component {
        render() {
          return <h1>Hello world!</h1>
        }
      }

      ReactDOM.render(<App />, document.getElementById('root'))
    </script>
  </body>
</html>

```

現在，如果您在瀏覽器中查看 `index.html`，您將看到我們建立的 `h1` 標記呈現給 DOM。



我們完成了這個 React 應用程式